GKE Deployment Runbook

Engineering Handbook · Internal · Owner: Platform Engineering

Purpose

This runbook is the prescriptive guide for deploying our Python/FastAPI microservices (api-service, auth-service, worker-service, ingestion-service, gateway, scheduler) onto GKE. It focuses on the three failure modes that cause most stalled or crash-looping rollouts: memory sizing, port alignment, and readiness probes. Follow it whenever you author or change a Deployment, its resource requests/limits, or its Service and probe configuration.

Standards

Memory requests and limits

Every container must declare CPU and memory `requests` and `limits`. Size limits from measured usage, not guesses. Undersized limits get the container OOMKilled (exit 137) under normal load. Keep `requests` close to `limits` for stable scheduling and predictable bin-packing, and add alerts when working-set memory approaches the limit.

```
resources:
 requests:
   cpu: "500m"
   memory: "512Mi"
 limits:
   cpu: "1"
   memory: "768Mi"   # set from observed peak + headroom, not a round guess
```

Measure real usage before setting numbers: `kubectl top pod` for a live snapshot, and container memory metrics over a representative load window. Investigate steady climbs (a leak or an unbounded in-process buffer) rather than only raising the ceiling.

Port alignment

The app's listen port, the `containerPort`, the probe port, and the Service `targetPort` must all be the same value. Bind to `0.0.0.0:$PORT` so the container is reachable from outside localhost.

```
ports:
 - containerPort: 8080
readinessProbe:
 httpGet:
   path: /healthz
   port: 8080
---
kind: Service
spec:
 ports:
   - port: 80
```

```
      targetPort: 8080
```

Readiness and liveness probes

Point readiness at a real health endpoint that returns 200 only when the app can serve traffic. Give it a sensible `initialDelaySeconds` and `timeoutSeconds`. Separate liveness (restart if wedged) from readiness (drain if degraded) so a slow-starting or degraded pod does not receive traffic prematurely.

```
livenessProbe:
 httpGet: { path: /livez, port: 8080 }
 initialDelaySeconds: 10
readinessProbe:
 httpGet: { path: /readyz, port: 8080 }
 initialDelaySeconds: 5
 timeoutSeconds: 2
```

Common failures and fixes

- Pod repeatedly OOMKilled (exit 137) and restarting → memory limit is below real usage; measure actual usage, set requests/limits accordingly, fix leaks or oversized buffers, and add memory-based alerts (BUG-1045).
- Pod stuck in CrashLoopBackOff and the LoadBalancer never goes healthy → container port mismatch; align `containerPort`, the server's `$PORT`, and the probe/Service `targetPort`, and bind `0.0.0.0:$PORT` (BUG-1007).
- New revision never becomes ready and the rollout stalls → readiness probe points at a path/port the app does not serve; point it at a real health endpoint on the correct port with a sane initialDelay/timeout and confirm it returns 200 (BUG-1048).
- Memory climbs steadily for hours then OOMs → unbounded in-process cache; bound it (LRU with max size and TTL) or move shared cache to Redis, and watch memory metrics.

Checklist

- Requests and limits set for CPU and memory, sized from measured peak plus headroom.
- Requests kept close to limits for stable scheduling.
- Listen port, `containerPort`, probe port, and Service `targetPort` all match.
- App binds `0.0.0.0:$PORT`, not `127.0.0.1`.
- Readiness probe hits a real endpoint returning 200; liveness and readiness are separate.
- Memory-based alerting configured; roll out and watch `kubectl rollout status`, restart counts, and pod memory.

Related incidents

BUG-1045, BUG-1007, BUG-1048, BUG-1051